

DATA 266 - Homework 4 Findings

Student: Nikhil Kanaparthy | SID4: 6441

1. Objective and data

This experiment built a character-level decoder-only GPT from basic PyTorch components. The supplied Shakespeare passage contained 1737 characters and a vocabulary of 49 unique characters. A sliding window of 128 tokens produced 1609 autoregressive input-target pairs.

2. Model and training

Setting	Value
Hidden dimension	128
Attention heads	4
Head dimension	32
Decoder blocks	2
Trainable parameters	425,777
Optimizer	Adam
Learning rate	0.0003
Batch size	32
Epochs	6

The attention layer manually projected and split Q, K, and V into four heads. A lower-triangular causal mask was applied before softmax. The measured maximum attention weight above the diagonal was 0.0000000000, confirming that future tokens were blocked.



Training loss decreased from 3.1579 to 2.1284 across 6 epochs.

3. Decoding comparison

Method	Setting	Unique bigram	Entropy	Average NLL
Greedy	argmax	0.0854	2.4054	1.4371
Temperature	T=0.5	0.3518	3.9600	1.5979
Temperature	T=1.0	0.5930	4.4829	2.1058
Temperature	T=1.5	0.7437	4.8625	2.8372
Top-k	k=5	0.3467	3.6981	1.7987
Top-k	k=20	0.5427	4.2803	1.9909

Greedy - argmax

ROMEO:
I the thathathat that t thathat that t thathat t that the that thathaname t t
that thathat that thathous t t t t t t the t t t t t the the the the the the the
the the the the the the the the

Temperature - T=0.5

ROMEO:
IEOMEOME be Romeoreanome t the thalll for ne whatheame thar thathameath R0:
He
whame thamy;
Bear thot t name, the f t anese he t he t ou the t the ise nome na nat ise
nar t whe whame ishere.
Roulll

Temperature - T=1.0

ROMEO:
Hew at hag?
Rhe.
Hee, stamel hysthe, ind Rze s t sheneco d me illal atome noun!
Sof.
JULI t bu thy helfut smy wens Ro it knghamanOrt har iveame;
Whese;
He nef w setot
the sETicoul she onan d s o;
I i

Temperature - T=1.5

ROMEO:
I Ronee
ccha beolcUgut fat stham e?
Wh hcha; so wouthy aoue n hortit tet ill
.:
Hef p
I?dor sd?l bat k!EOMoseshes? athDelo,
N.
TCathethomy METu?
JST!
Barl wrt? nofase;
JDeht t st isze;veul?T:
T:OWh n

Top-k - k=5

ROMEO: aknathe the is than, ndore se n atothe ithe t t the whor t sperert nguthat se sis
whe is ant ser inoutoutand nofat st ithall noul s t t s s nd and, ndoreshe is the t she
ndomardellore ald tind shes

Top-k - k=20

```
ROMEO:  
By al f thanak d and l methe we  
JULI me b:  
Whalll bewhy no, I sthome buscene,,  
Whe ks orereakmy se ashad but bares aitthe ithathe isame o me ar eof nonal d, es;  
I  
thapo?  
The ant tho ber, ir d t ino,
```

4. Analysis and conclusion

Greedy decoding chooses the single highest-probability character at every step. It is deterministic and usually follows the model's strongest learned patterns, but it can repeat a word, phrase, or punctuation pattern because it never explores an alternative.

Temperature sampling draws from the complete probability distribution after dividing the logits by a temperature. At $T=0.5$ the distribution is sharper and the output stays relatively conservative. $T=1.0$ uses the model's original distribution. At $T=1.5$ the distribution is flatter, so unusual characters are more likely and diversity increases, but local coherence can decrease.

Top-k sampling first limits the choice to the k most probable characters. With $k=5$ the shortlist is restrictive and tends to preserve familiar patterns. With $k=20$ the model has a larger set of possible characters, producing more variety with a greater risk of awkward transitions.

For this run, Greedy (argmax) had the lowest average NLL (1.4371), so it was the most coherent according to the model-probability proxy. Temperature ($T=1.5$) had the largest combined diversity score (1.6097), so it was the most diverse output. Overall, greedy or conservative sampling is preferable when consistency is the main goal, while higher-temperature or larger-k sampling is preferable when variety is more important.

References

Vaswani et al. (2017), Attention Is All You Need. PyTorch documentation for Linear, LayerNorm, GELU, Adam, and tensor operations. Course Demo 4: Building a Tiny GPT.